4. Database Security

4.1 Database Security: Security Requirements

Database security is a comprehensive set of policies, technologies, and administrative controls designed to protect database systems from unauthorized access, misuse, data leakage, corruption, and destruction. In modern organizations, databases store highly valuable information such as customer records, financial transactions, employee data, healthcare records, research documents, and intellectual property. Because data is considered a strategic asset, protecting it is essential not only for operational continuity but also for legal compliance, organizational reputation, and customer trust.

A secure database system must satisfy several fundamental security requirements. These requirements form the foundation of database protection mechanisms and are implemented through technical, administrative, and physical controls.

---

1. Confidentiality

Confidentiality ensures that sensitive information stored in the database is accessible only to authorized users and remains protected from unauthorized disclosure. It prevents data exposure to hackers, malicious insiders, competitors, or unauthorized third parties.

In database systems, confidentiality is maintained through multiple layers of protection:

- **Authentication mechanisms** such as usernames, passwords, multi-factor authentication (MFA), and biometric verification ensure that only legitimate users gain access.
- **Access control mechanisms** such as Role-Based Access Control (RBAC) restrict users to only the data necessary for their role.
- **Encryption techniques** protect data both at rest (stored data) and in transit (data transmitted over networks).

- **Views and data masking** limit exposure of sensitive fields such as social security numbers or credit card details.

For example, in a banking system, customer account details, transaction history, and loan records must be accessible only to authorized bank employees. If confidentiality is breached, it may result in financial fraud, identity theft, or legal penalties under data protection regulations.

Thus, confidentiality protects privacy and prevents data leakage.

---

2. Integrity

Integrity ensures that the data stored in the database remains accurate, consistent, and reliable throughout its lifecycle. It guarantees that information is not altered improperly, whether by accident, system error, or malicious activity.

Data integrity operates at multiple levels:

- **Entity integrity** ensures that each record is uniquely identifiable, typically enforced using primary keys.
- **Referential integrity** ensures logical relationships between tables through foreign keys.
- **Domain integrity** restricts data values to valid ranges or formats using check constraints and data types.
- **Transaction integrity** ensures that database operations follow ACID (Atomicity, Consistency, Isolation, Durability) properties.

Integrity controls prevent unauthorized modification and accidental corruption. For example, in a banking system, the account balance should only change through legitimate transactions such as deposits or withdrawals. If unauthorized modification occurs, financial discrepancies and trust issues may arise.

Maintaining integrity ensures that decision-making processes based on database information remain accurate and dependable.

---

3. Availability

Availability ensures that authorized users can access the database and its services whenever needed. A database that is secure but inaccessible is not useful. Therefore, systems must be designed to minimize downtime and ensure continuous operation.

Threats to availability include:

- Denial-of-Service (DoS) attacks
- Hardware failures
- Power outages
- Software bugs
- Natural disasters

To ensure high availability, organizations implement:

- **Regular backups** to restore lost data
- **Replication** to maintain multiple copies of data
- **Clustering and load balancing** to distribute workload
- **Disaster recovery plans** to restore operations after catastrophic events
- **Failover mechanisms** to switch to standby systems automatically

For example, in an e-commerce platform, if the database becomes unavailable during peak shopping hours, it may result in revenue loss and customer dissatisfaction. Therefore, availability is critical for business continuity.

---

4. Authentication and Authorization

Authentication and authorization are core mechanisms that control access to database systems.

Authentication

Authentication verifies the identity of users attempting to access the database. It answers the question: *“Who are you?”*

Common authentication methods include:

- Username and password
- Multi-factor authentication (MFA)
- Smart cards
- Biometric authentication
- Digital certificates

Strong authentication prevents unauthorized individuals from entering the system.

Authorization

Authorization determines what actions an authenticated user is permitted to perform. It answers the question: *“What are you allowed to do?”*

Authorization mechanisms include:

- Granting and revoking privileges
- Role-based access control (RBAC)
- Attribute-based access control (ABAC)
- Fine-grained access policies

For example, a database administrator may have full privileges (CREATE, DROP, ALTER), while a data entry operator may only have INSERT and SELECT permissions. Proper authorization enforces the principle of least privilege, meaning users are given only the permissions necessary to perform their tasks.

Together, authentication and authorization ensure controlled and structured access to database resources.

---

5. Accountability and Auditing

Accountability ensures that every action performed within the database can be traced back to a specific user or process. It discourages misuse because users know their actions are being monitored.

Auditing mechanisms record:

- Login attempts (successful and failed)
- Data modifications (INSERT, UPDATE, DELETE)
- Privilege changes
- Access to sensitive tables
- Administrative activities

Audit logs help in:

- Detecting suspicious activities
- Investigating security breaches
- Meeting regulatory compliance requirements
- Performing forensic analysis after incidents

For example, if confidential employee salary data is leaked, audit logs can help identify who accessed or exported the data and at what time. This strengthens internal security controls and ensures responsibility for actions.

Conclusion

The security requirements of a database system—Confidentiality, Integrity, Availability, Authentication, Authorization, Accountability, and Auditing—form the foundation of database protection strategies. These requirements work together to ensure that data remains private, accurate, accessible, and traceable.

In modern information systems, where data is a critical organizational asset, implementing strong database security controls is not optional but essential for operational success, legal compliance, and long-term sustainability.

4.2 Reliability and Integrity

Reliability in Database Systems

Reliability refers to the ability of a database system to operate correctly and continuously over a long period of time, even in the presence of hardware failures, software bugs, power outages, network disruptions, or human errors. A reliable database system guarantees that once a transaction is committed, its results will not be lost, and incomplete transactions will not corrupt the system.

In real-world applications such as banking systems, hospital management systems, airline reservation systems, and e-commerce platforms, reliability is critical because data loss or inconsistency can result in financial loss, legal issues, or even risk to human life.

A reliable database system ensures:

- Correct execution of transactions
- Protection against data loss
- Quick recovery from failures
- Continuous availability of services

Reliability is closely associated with the **ACID properties** of transactions:

- **Atomicity** – A transaction either completes fully or does not execute at all.
- **Consistency** – The database remains in a valid state before and after the transaction.
- **Isolation** – Concurrent transactions do not interfere with each other.
- **Durability** – Once committed, changes remain permanent even after system failure.

These properties collectively guarantee dependable transaction processing.

Mechanisms to Achieve Database Reliability

1. Transaction Management

Transaction management ensures that database operations are executed in a controlled and consistent manner. A transaction is a sequence of one or more SQL operations treated as a single logical unit of work.

For example, in a fund transfer operation:

- Debit amount from Account A
- Credit amount to Account B

If the system crashes after debiting but before crediting, transaction management ensures that the entire transaction is rolled back, preventing data inconsistency.

Transaction management mechanisms include:

- Commit and rollback operations
- Concurrency control techniques (locking, timestamping)
- Logging mechanisms

Without proper transaction management, databases would frequently end up in inconsistent states.

---

2. Backup and Recovery Mechanisms

Backup and recovery mechanisms protect the database against data loss due to hardware failure, accidental deletion, cyberattacks, or disasters.

Backup involves creating copies of data at regular intervals. Types of backups include:

- Full backup
- Incremental backup
- Differential backup

Recovery involves restoring the database to a consistent state using:

- Transaction logs
- Checkpoints
- Backup files

For example, if a server crashes at 3 PM, recovery mechanisms can restore the database to the last consistent state before the crash.

Backup and recovery strategies are essential for business continuity and disaster recovery planning.

---

3. Redundancy and Replication

Redundancy refers to storing duplicate copies of data to prevent data loss. Replication involves maintaining synchronized copies of the database on multiple servers.

Types of replication:

- Master-slave replication
- Multi-master replication
- Synchronous replication
- Asynchronous replication

Replication provides:

- High availability
- Load balancing

- Fault tolerance
- Disaster recovery

If one server fails, another replica can continue serving users, ensuring uninterrupted service.

---

4. Fault Tolerance Systems

Fault tolerance allows a database system to continue functioning even when some components fail.

Fault tolerance is achieved through:

- RAID storage systems
- Clustered database servers
- Automatic failover mechanisms
- Distributed database architectures

For example, in a clustered system, if one node fails, another node automatically takes over, minimizing downtime.

Fault tolerance improves system availability and reduces service interruption.

Integrity in Database Systems

Integrity refers to the correctness, accuracy, and consistency of data stored in a database. It ensures that the data remains valid according to predefined rules and constraints.

While reliability ensures that the system works properly, integrity ensures that the data inside the system is meaningful and correct.

Integrity constraints are rules enforced by the database management system (DBMS) to prevent invalid data entry.

Types of Data Integrity

1. Entity Integrity

Entity integrity ensures that each table has a primary key that uniquely identifies every record. The primary key must be:

- Unique
- Not NULL

For example, in a Student table:

- Student_ID acts as a primary key.
- No two students can have the same Student_ID.
- Student_ID cannot be empty.

Entity integrity prevents duplicate and unidentified records in a table.

---

2. Referential Integrity

Referential integrity ensures that relationships between tables remain consistent.

A foreign key in one table must match a primary key in another table or be NULL (if allowed).

Example:

- Customer table (Customer_ID as primary key)
- Order table (Customer_ID as foreign key)

An order cannot exist without a valid customer. This prevents orphan records and maintains logical consistency.

Referential integrity can enforce actions such as:

- ON DELETE CASCADE
- ON UPDATE CASCADE
- RESTRICT
- SET NULL

These rules define what happens when referenced data is modified or deleted.

---

3. Domain Integrity

Domain integrity ensures that the values entered in a column conform to predefined rules such as:

- Data type (integer, string, date)
- Length restrictions
- Format constraints
- Range limits
- Default values

For example:

- Age must be between 0 and 120
- Email must follow a valid format
- Salary must be numeric

Domain integrity prevents invalid or meaningless data from entering the system.

---

4. User-Defined Integrity

User-defined integrity refers to custom business rules defined by the organization that are not covered by basic integrity constraints.

Examples:

- Salary must be greater than zero
- Loan amount cannot exceed credit limit
- Employee retirement age must be 60

These rules are implemented using:

- CHECK constraints
- Triggers
- Stored procedures
- Application-level validations

User-defined integrity ensures that the database follows organizational policies and business logic.

Relationship Between Reliability and Integrity

Reliability and integrity are complementary concepts:

- Reliability ensures that data is not lost and the system operates correctly.
- Integrity ensures that stored data is accurate and valid.

A database may be reliable but contain incorrect data if integrity rules are weak. Similarly, a database with strong integrity constraints is useless if it frequently crashes and loses data.

Therefore, both reliability and integrity are essential for building a trustworthy database system. They form the foundation of secure, stable, and dependable information systems used in modern organizations.

4.3 Sensitive Data

Sensitive data refers to any information that must be protected from unauthorized access, disclosure, alteration, or destruction because of its confidential, private, or critical nature. The exposure of such data can result in serious consequences including financial loss, identity theft, reputational damage, operational disruption, and legal penalties. In modern information systems, especially databases and networked environments, protecting sensitive data is a fundamental requirement of information security.

Sensitive data is closely related to the core security principles of confidentiality, integrity, and availability (CIA triad). Confidentiality ensures that only authorized users can access the data, integrity ensures that the data is not modified improperly, and availability ensures that the data is accessible to authorized users when required.

Types of Sensitive Data

Sensitive data can be classified into several categories depending on its nature and impact if disclosed.

1. Personal Information

Personal information refers to data that can identify an individual either directly or indirectly. This includes:

- Name
- Address
- Phone number
- Email address
- Date of birth

- National identification numbers

This type of data is often referred to as Personally Identifiable Information (PII). If attackers gain access to personal information, they can perform identity theft, fraud, or social engineering attacks. Organizations such as banks, universities, hospitals, and e-commerce platforms collect large amounts of personal information, making them attractive targets for cybercriminals.

---

2. Financial Data

Financial data includes information related to monetary transactions and financial accounts. Examples include:

- Bank account numbers
- Credit and debit card details
- Transaction history
- Salary records
- Tax information

Financial data is highly valuable in the underground cybercrime market. Unauthorized disclosure can lead to direct financial theft, fraudulent transactions, and legal liability for organizations. For example, if a payment gateway database is compromised, attackers may steal thousands of credit card numbers, resulting in massive financial and reputational damage.

---

3. Medical Records

Medical records contain highly sensitive health-related information about individuals, such as:

- Medical history
- Diagnosis reports
- Laboratory test results
- Prescriptions
- Insurance details

Healthcare data is extremely sensitive because it reveals intimate details about a person's physical and mental health. Unauthorized disclosure may lead to discrimination, blackmail, or emotional distress. Hospitals and healthcare systems must implement strict data protection controls to maintain patient confidentiality.

---

4. Business Secrets and Intellectual Property

Business organizations store proprietary information that gives them competitive advantage. This includes:

- Trade secrets
- Product designs
- Research and development data
- Source code
- Business strategies
- Customer databases

If competitors or attackers obtain this information, it can cause significant economic loss. For example, leakage of software source code can expose vulnerabilities or allow competitors to copy the product. Protecting intellectual property is therefore critical for long-term business sustainability.

---

5. Government Classified Data

Government agencies manage classified information related to:

- National security
- Military operations
- Intelligence reports
- Diplomatic communications

Unauthorized access to classified data can threaten national security and public safety. Governments typically use multilevel security models and strict access control mechanisms to ensure that only authorized personnel can access such data based on clearance levels.

Protection Mechanisms for Sensitive Data

To protect sensitive data, organizations implement multiple layers of technical, administrative, and physical security controls.

---

1. Data Encryption

Encryption is one of the most effective methods to protect sensitive data.

- **Encryption at rest** protects data stored in databases, hard drives, or cloud storage. Even if attackers gain physical access to storage devices, they cannot read the encrypted data without the decryption key.
- **Encryption in transit** protects data while it is being transmitted over networks using secure protocols such as TLS/SSL. This prevents interception through attacks like packet sniffing or man-in-the-middle attacks.

Strong encryption algorithms and proper key management practices are essential to ensure effectiveness.

---

2. Data Masking

Data masking involves hiding or obfuscating sensitive data so that unauthorized users cannot see the actual values. For example:

- Displaying a credit card number as **** **** **** 1234
- Replacing real names with pseudonyms in testing environments

Data masking is particularly useful in development and testing environments where real production data should not be exposed.

---

3. Access Control Policies

Access control ensures that only authorized users can access sensitive data. This is implemented using:

- Authentication mechanisms (passwords, biometrics, multi-factor authentication)
- Authorization models (Role-Based Access Control – RBAC, Mandatory Access Control – MAC)
- Principle of least privilege (users are given only the minimum permissions required to perform their tasks)

Proper access control reduces the risk of insider threats and accidental data leakage.

---

4. Data Classification

Data classification is the process of categorizing data based on its sensitivity level. Common classification levels include:

- Public
- Internal
- Confidential
- Highly Confidential / Restricted

By classifying data, organizations can apply appropriate security controls according to the sensitivity level. For example, highly confidential data may require encryption, strict access control, and continuous monitoring.

---

5. Secure Storage and Backup

Sensitive data must be stored securely using:

- Secure database configurations
- Hardened servers
- Regular security patching
- Backup encryption

Regular backups are essential to ensure data availability in case of hardware failure, ransomware attacks, or natural disasters. However, backups themselves must also be protected to prevent misuse.

Legal and Regulatory Compliance

Organizations must comply with data protection laws and regulations to ensure proper handling of sensitive information. These regulations define how data should be collected, stored, processed, and shared. Non-compliance may result in:

- Heavy financial penalties
- Legal action
- Suspension of business operations
- Loss of customer trust

Examples of regulatory requirements include:

- Data minimization (collect only necessary data)
- User consent for data processing
- Right to access and correct personal data
- Breach notification requirements

Compliance is not only a legal obligation but also a best practice for maintaining trust and credibility.

Conclusion

Sensitive data is a critical asset in any organization. Its protection is essential to prevent financial losses, legal consequences, operational disruption, and reputational damage. By implementing strong encryption, access control, data masking, classification policies, and secure storage practices, organizations can significantly reduce the risk of data breaches. Effective protection of sensitive data requires a combination of technology, policy, legal compliance, and continuous monitoring to address evolving cyber threats.

4.4 Inference Problem

The **inference problem** in database security arises when a user is able to deduce or derive sensitive information from data that is otherwise considered non-sensitive. Even though direct access to confidential data may be restricted through access control mechanisms, a user may still obtain that information indirectly by intelligently analyzing available data. This makes inference a subtle and dangerous threat because it does not involve unauthorized access in the traditional sense; instead, it exploits legitimate access to derive protected information.

In simple terms, inference occurs when:

- A user has access to multiple pieces of non-sensitive data.
- By combining or analyzing those data items, the user can logically deduce sensitive information.
- The database system does not directly reveal the confidential value, but it can be computed indirectly.

For example, consider a database in an organization where:

- Individual salaries are confidential.
- The average salary of a department is accessible.
- Salaries of all employees except one are accessible.

If a department has 5 employees and the average salary is known, and the salaries of 4 employees are known, the salary of the fifth employee can easily be calculated:

Restricted Salary = (Average × Total Employees) − Sum of Known Salaries

This is a classic example of an inference attack.

---

1. Why the Inference Problem is Serious

The inference problem is particularly serious because:

1. **It bypasses traditional access controls** The user does not violate access rules. They only use permitted queries and permitted data.

2. **It is difficult to detect** The system may not recognize that a user is performing calculations to deduce sensitive data.

3. **It threatens privacy and confidentiality** Inference can expose personal information such as:

    - Salaries
    - Medical conditions
    - Criminal records
    - Military information
    - Financial details

4. **It is common in statistical databases** Statistical databases provide aggregate data (like averages, totals, percentages). These aggregates may unintentionally reveal individual values.

Because of these reasons, inference control is a major concern in high-security environments such as government systems, healthcare systems, census databases, and research databases.

---

2. Inference in Statistical Databases

A statistical database provides summary information rather than individual records. For example:

- Average income of employees in a department
- Number of patients with a certain disease
- Percentage of people in a region with a specific qualification

Although individual records are not shown, problems arise when:

- The group size is very small (e.g., 1 or 2 individuals).
- Multiple overlapping queries are allowed.
- Users can repeatedly query slightly different subsets of data.

Example:

Query 1: Average salary of all employees in Department A = 50,000 Query 2: Average salary of Department A excluding Mr. X = 45,000

From these two queries, Mr. X's salary can be inferred.

This type of attack is known as a **difference attack**.

---

3. Types of Inference Attacks

Inference attacks can be categorized as follows:

1. **Direct Inference** Sensitive data is directly deduced from aggregate values.

2. **Indirect Inference** Sensitive information is derived by combining multiple query results.

3. **Tracker Attacks** The attacker creates specially designed query sets to isolate the value of a specific individual.

4. **Data Correlation Attacks** Combining data from multiple sources or databases to infer hidden information.

These attacks exploit logical reasoning rather than system vulnerabilities.

---

4. Inference Control Techniques

To prevent inference attacks, database systems use several control techniques.

4.1 Data Suppression

Data suppression involves hiding or not displaying specific sensitive values.

- Individual records are not shown.
- Small group statistics are suppressed.
- Certain attributes are completely removed from query results.

For example: If a query result involves fewer than 3 employees, the system may refuse to provide the result.

Advantages:

- Simple to implement.
- Effective for small groups.

Limitations:

- May reduce data usability.
- Too much suppression reduces the usefulness of the database.

4.2 Query Restriction

Query restriction limits the type or number of queries a user can perform.

Methods include:

- Restricting queries on small data sets.
- Limiting overlapping queries.
- Monitoring repeated similar queries.
- Limiting the number of queries per user.

For example: The system may block queries where:

- The result set size is below a threshold (e.g., less than 5 records).
- The query is too similar to a previous query.

Advantages:

- Prevents difference attacks.
- Reduces repeated inference attempts.

Limitations:

- May inconvenience legitimate users.
- Complex to design properly.

4.3 Data Perturbation

Data perturbation means adding small random noise to the data before presenting it.

For example:

- Instead of showing exact average salary as 50,000, show 49,800 or 50,200.
- Randomly adjust results within a controlled range.

This makes it difficult to calculate exact individual values.

Advantages:

- Maintains overall statistical usefulness.
- Protects individual privacy.

Limitations:

- Reduces accuracy.
- Excessive noise may distort research results.

This concept is widely used in modern privacy-preserving techniques such as differential privacy.

4.4 Limiting Aggregate Queries

Aggregate queries (SUM, AVG, COUNT, MIN, MAX) are the primary tools used in inference attacks. Therefore:

- The system may restrict the number of aggregate queries.
- It may enforce a minimum group size.
- It may restrict the use of complementary queries (e.g., including and excluding one individual).

For example:

- Only allow aggregate queries if the result is based on at least 5 records.
- Prevent querying of overlapping subsets that could isolate individuals.

Advantages:

- Directly addresses the main source of inference attacks.
- Protects small group data.

Limitations:

- Limits analytical flexibility.

---

5. Inference in Multilevel Security Systems

In multilevel secure databases (e.g., military systems):

- Data is classified at different levels: Confidential, Secret, Top Secret.
- Users are allowed to access data only at or below their clearance level.

Even in such systems, inference is possible. For example:

A user with Confidential clearance may combine multiple low-level data items and infer a Top Secret fact.

Therefore, inference control is essential in multilevel database systems to maintain strict separation of classified information.

---

6. Challenges in Inference Control

Inference control is complex because:

- It is difficult to predict all possible logical deductions.
- Strict controls reduce database usability.
- There is a trade-off between privacy and data utility.
- Users may use external information combined with database data.

Thus, database designers must carefully balance:

Security + Privacy + Accuracy + Usability

---

7. Conclusion

The inference problem represents a major security threat in database systems. It demonstrates that simply restricting direct access to sensitive data is not enough. Even non-sensitive data, when combined intelligently, can reveal confidential information.

Inference control techniques such as data suppression, query restriction, data perturbation, and limiting aggregate queries are essential, especially in statistical databases, government systems, healthcare databases, and multilevel security environments.

Understanding the inference problem is crucial for designing secure database systems that protect confidentiality while still providing meaningful and useful information to authorized users.

4.5 Multilevel Database

A **multilevel database (MLDB)** is a database system designed to store and manage information that exists at different security classification levels within the same database environment. It is primarily used in environments where data sensitivity varies significantly, such as military, defense, intelligence agencies, and government organizations.

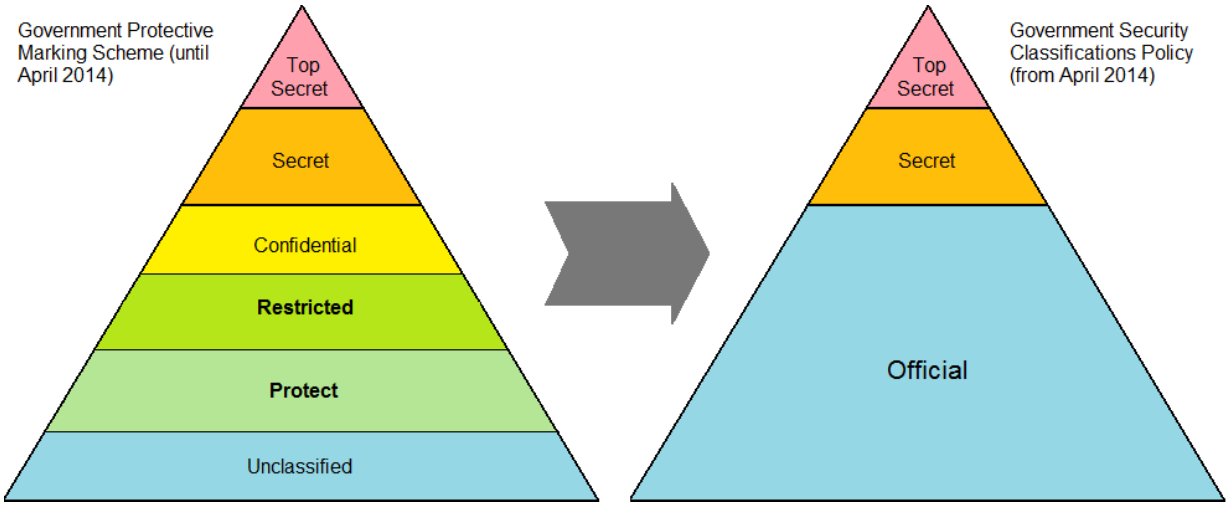
In such environments, information is categorized into different classification levels, for example:

- **Unclassified**
- **Confidential**

- **Secret**
- **Top Secret**

Each piece of data in the database is assigned a **security label**, and each user is assigned a **security clearance level**. The system ensures that users can only access data that matches their authorized clearance level.

The main goal of a multilevel database is to **prevent unauthorized disclosure of sensitive information while still allowing controlled sharing of less sensitive data within the same system**.



Multilevel security follows the principles of Mandatory Access Control (MAC), where access decisions are based on security labels rather than user discretion.

Key principles include:

- No Read Up (a user cannot read data at a higher security level)
- No Write Down (a user cannot write data to a lower security level)

These principles are derived from the Bell-LaPadula Model, which focuses on maintaining confidentiality in secure systems.

4.5.1 Need for Multilevel Databases

In many organizations, especially military and intelligence sectors, information is not uniformly sensitive. For example:

- Public announcements may be unclassified.
- Operational details may be confidential.
- Strategic defense plans may be top secret.

If separate databases are maintained for each classification level, it becomes difficult to manage, expensive to maintain, and inefficient for cross-level operations.

A multilevel database solves this problem by:

- Allowing storage of all classification levels in a single database system.
- Enforcing strict access control policies automatically.
- Reducing redundancy and administrative complexity.
- Supporting controlled information flow between different levels.

Thus, MLDB provides both **security and operational efficiency**.

4.6 Proposals for Multilevel Security

Multilevel Security (MLS) in databases is designed to handle information that exists at different classification levels such as Unclassified, Confidential, Secret, and Top Secret. The primary objective is to ensure that users can access only the data for which they have proper clearance, while preventing both direct and indirect leakage of higher-level information. Over the years, several architectural and design proposals have been introduced to implement MLS in database systems. Each approach addresses specific security challenges such as inference, covert channels, integrity protection, and access control enforcement.

---

1. Trusted Subject Approach

The Trusted Subject Approach introduces special programs or system components known as *trusted subjects* that are authorized to access and manage data across multiple security levels. Unlike ordinary users, trusted subjects are permitted to bypass certain mandatory access control (MAC) restrictions, but they are carefully designed, verified, and audited to ensure that they do not violate security policies.

In a typical multilevel database, ordinary users are restricted by strict rules such as “no read up” and “no write down” (as defined in models like the Bell-LaPadula model). However, some administrative or operational tasks require interaction between different classification levels. For example, generating a summary report that combines Secret and Confidential data may require access to multiple levels. In such cases, a trusted subject program performs the operation securely and ensures that the output is properly sanitized before being released to lower-level users.

The key characteristics of this approach include:

- Strict certification and verification of trusted programs.
- Auditing and logging of all actions performed by trusted subjects.
- Minimal number of trusted components to reduce security risk.

The advantage of this method is flexibility in handling cross-level operations. However, the system’s security heavily depends on the correctness and trustworthiness of the trusted subject. If compromised, it can become a major vulnerability.

---

2. Kernelized Architecture

Kernelized Architecture places all security enforcement mechanisms inside a secure and minimal core component called the *security kernel*. This kernel is responsible for enforcing mandatory access control policies and ensuring complete mediation of every access request.

The idea is inspired by secure operating system designs, where a small and formally verified kernel controls all access to system resources. In a database context, the kernel enforces rules such as:

- Checking user clearance before granting access.
- Controlling data labeling and classification.
- Preventing unauthorized modifications.
- Ensuring data integrity and confidentiality.

Since the kernel is small and isolated from other system components, it can be rigorously tested and verified. Regular users and application programs cannot modify or bypass it. This reduces the possibility of security policy violations.

Advantages of kernelized architecture include:

- Strong and centralized enforcement of security policies.
- Easier formal verification due to smaller trusted computing base (TCB).
- Reduced risk of accidental or malicious policy modification.

However, this approach may introduce performance overhead because every database access must pass through the security kernel. Additionally, designing and verifying a secure kernel is complex and resource-intensive.

---

3. Polyinstantiation

Polyinstantiation is a unique mechanism used in multilevel databases to prevent inference attacks. It allows the existence of multiple tuples (records) with the same primary key but different security classifications.

In a traditional database, a primary key uniquely identifies a record. However, in a multilevel environment, strict uniqueness may reveal the presence of higher-level information. For example:

- A Secret-level user inserts a record with EmployeeID = 101.
- A Confidential-level user attempts to insert a record with the same EmployeeID.
- If the system rejects the insertion due to duplication, the lower-level user may infer that a Secret-level record already exists.

To prevent this inference, the system allows multiple records with the same primary key but different security labels. Each user sees only the version of the record that matches their clearance level.

Polyinstantiation provides:

- Protection against inference attacks.
- Preservation of data confidentiality.
- Logical separation of data across levels.

However, it increases database complexity and may cause consistency management challenges. Careful design is required to maintain data integrity while supporting multiple instances.

---

4. View-Based Security

View-Based Security restricts user access through customized database views. A view is a virtual table derived from one or more base tables, presenting only selected data to users.

In a multilevel security environment, different users are granted access to different views based on their clearance. For example:

- A Confidential-level user may see only basic employee details.
- A Secret-level user may see additional salary or project information.
- A Top Secret-level user may access complete operational data.

By carefully designing views, the database administrator ensures that:

- Users cannot access unauthorized columns or rows.
- Sensitive attributes are hidden from lower-level users.
- Aggregated or filtered information prevents inference.

This approach enhances usability because users interact with the database normally without being aware of hidden data. It also simplifies implementation by leveraging standard database features such as SQL views.

However, view-based security alone may not be sufficient for strong MLS enforcement unless combined with mandatory access control mechanisms. It is generally used as a complementary strategy rather than a standalone solution.

---

5. Encryption-Based Approach

The Encryption-Based Approach secures data by encrypting it according to its classification level. Users are provided cryptographic keys corresponding to their security clearance. Only users with appropriate keys can decrypt and read the data.

In this approach:

- Confidential data may be encrypted with one key.
- Secret data with another key.
- Top Secret data with a stronger or separate key.

Even if an unauthorized user gains physical access to the database files, they cannot interpret the data without the correct decryption keys. Encryption can be applied at:

- Column level (sensitive attributes).
- Row level (classified records).
- Entire database level.

This approach provides strong confidentiality and protects against insider threats and external breaches. It is particularly useful in distributed systems and cloud environments where storage security is a concern.

However, encryption introduces challenges such as:

- Key management complexity.
- Performance overhead during encryption and decryption.
- Difficulty in performing certain database operations on encrypted data.

Despite these challenges, encryption remains a critical component of modern multilevel database security strategies.

Conclusion

The proposals for multilevel security aim to ensure strict separation of classified information while maintaining database usability and efficiency. The Trusted Subject Approach enables controlled cross-level operations. Kernelized Architecture enforces centralized and secure policy control. Polyinstantiation prevents inference attacks. View-Based Security restricts user visibility through logical abstraction. Encryption-Based Approaches protect data confidentiality even in case of unauthorized access.

In practical systems, these approaches are often combined to provide comprehensive multilevel security. By integrating architectural control, access restrictions, data labeling, and cryptographic protection, modern database systems can effectively support secure information management across multiple classification levels.

Security in Network

4.7 Threats in Network

A network connects multiple computers and devices, enabling communication and data exchange. However, it is vulnerable to various threats.

Common network threats include:

1. Eavesdropping Unauthorized interception of communication to steal sensitive information.

2. Spoofing Attacker pretends to be a legitimate user or system.

3. Denial-of-Service (DoS) Attacks Overwhelming a system with excessive traffic to make it unavailable.

4. Man-in-the-Middle (MITM) Attack Attacker secretly intercepts and alters communication between two parties.

5. Malware and Ransomware Malicious software that damages or encrypts systems.

6. Phishing Fraudulent emails or websites used to steal credentials.

Understanding threats helps in designing effective countermeasures.

4.8 Network Security Controls

Network security controls are measures implemented to protect network infrastructure and data.

They include:

1. Administrative Controls Security policies, training programs, and access management procedures.

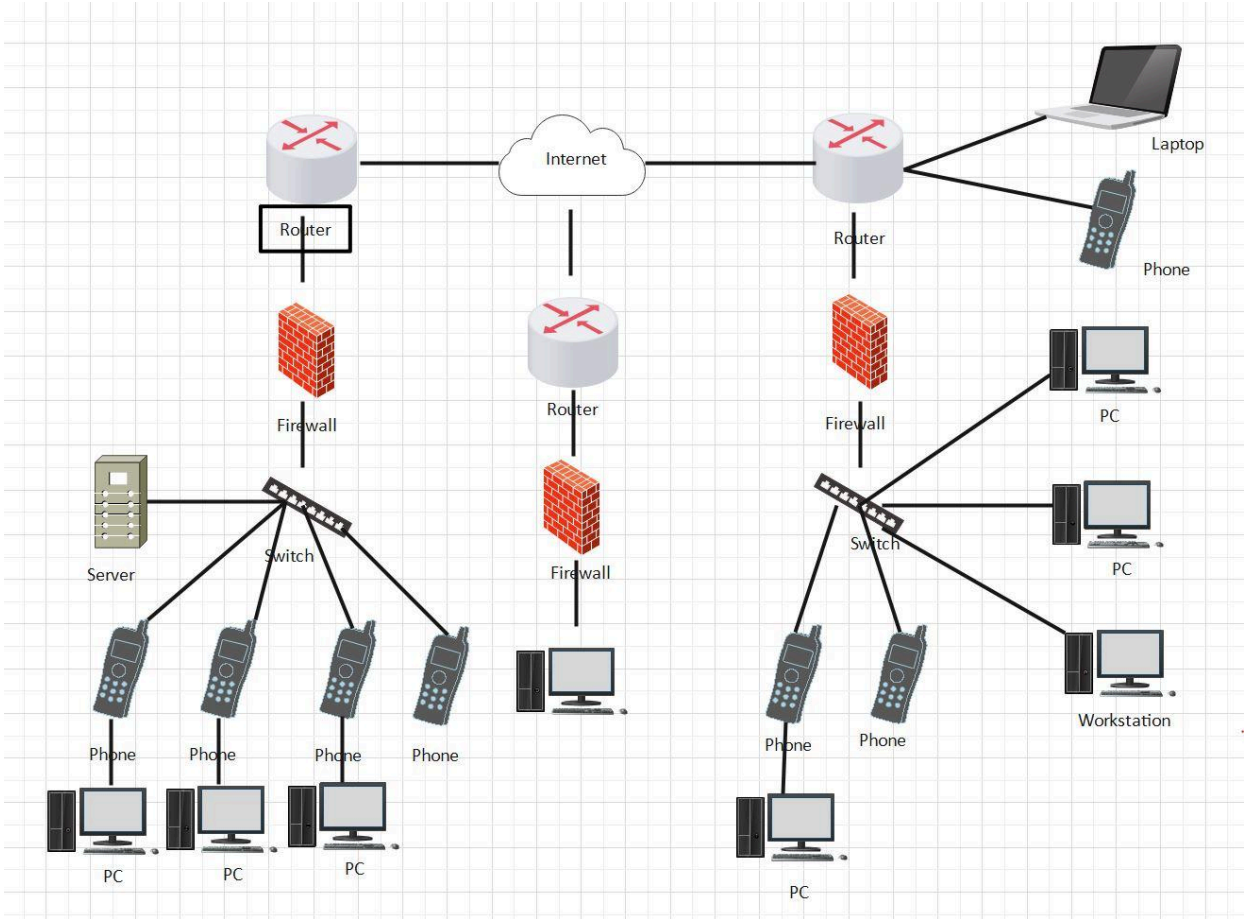
2. Technical Controls Encryption, secure protocols (HTTPS, SSL/TLS), antivirus software, intrusion detection systems.

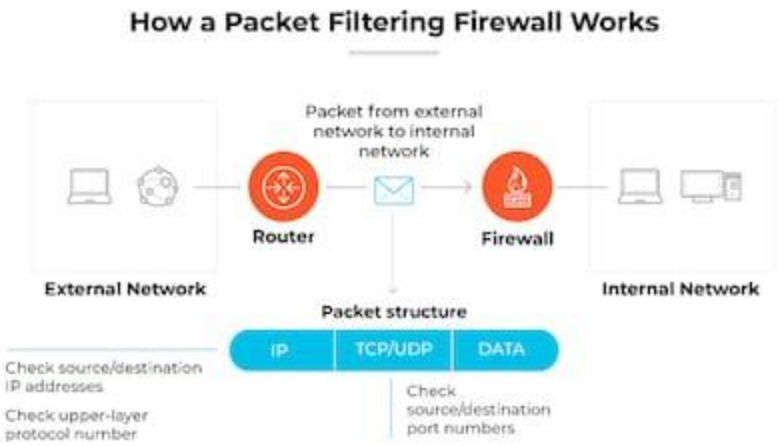
3. Physical Controls Securing servers, routers, and network devices from physical access.

Layered security (Defense-in-Depth) is used to provide multiple protection layers.

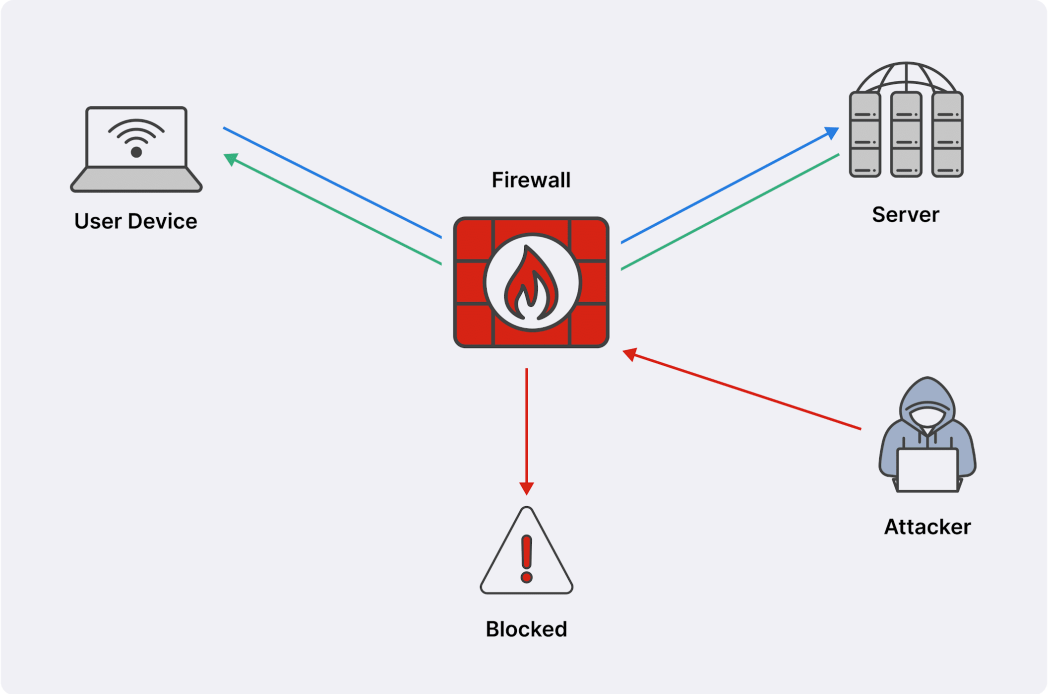
4.9 Firewalls

A firewall is a security device that monitors and controls incoming and outgoing network traffic based on predefined security rules.





How Does a Stateful Firewall Work?



Types of firewalls include:

1. Packet Filtering Firewall Examines packets based on IP address, port number, and protocol.

2. Stateful Inspection Firewall Tracks active connections and makes decisions based on connection state.

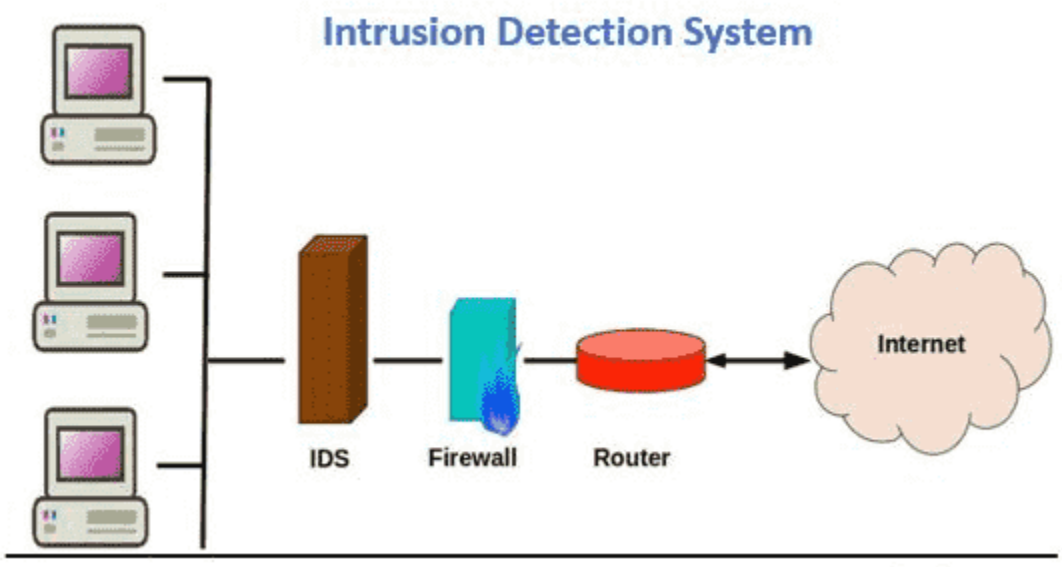
3. Application-Level Firewall (Proxy Firewall) Filters traffic at the application layer, such as HTTP or FTP.

4. Next-Generation Firewall (NGFW) Includes deep packet inspection, intrusion prevention, and application awareness.

Firewalls act as a barrier between trusted internal networks and untrusted external networks.

4.10 Intrusion Detection Systems (IDS)

An Intrusion Detection System monitors network or system activities for malicious activities or policy violations.



Types of IDS:

1. Network-Based IDS (NIDS) Monitors network traffic for suspicious activity.

2. Host-Based IDS (HIDS) Monitors activities on a single device.

IDS can be:

- Signature-Based (detects known attack patterns)
- Anomaly-Based (detects unusual behavior)

An Intrusion Prevention System (IPS) can automatically block detected threats.

4.11 Secure E-Mail

Email communication is widely used but vulnerable to attacks such as phishing, spoofing, and malware.

Secure email mechanisms include:

1. Encryption Ensures confidentiality of email content using public key cryptography.

2. Digital Signatures Verifies sender authenticity and message integrity.

3. Secure Email Protocols Protocols like SSL/TLS secure email transmission.

4. Spam Filters and Anti-Phishing Mechanisms Detect and block malicious emails.

Two widely used secure email technologies are:

- Pretty Good Privacy (PGP)
- S/MIME

Secure email ensures confidentiality, integrity, authentication, and non-repudiation of electronic communication.
